

计算机视觉第二次作业——图像分类系统

一、作业要求与完成情况

1. 作业要求

本次作业要求编写一个 15-Scene 数据集图像分类系统，能够对输入图像进行类别预测。应用 SIFT 特征、Kmeans 聚类、词袋表示和支撑向量机等技术。

2. 完成情况

本次作业已完成一个符合作业要求的模块化图像分类系统，实现了多种实验配置：

1. SIFT 特征+Kmeans+词袋表示+支撑向量机（本实验中作为 baseline）
2. 密集 SIFT+Kmeans+词袋表示+支撑向量机
3. SIFT 特征+Kmeans+词袋表示+空间金字塔匹配+支撑向量机
4. SIFT 特征+RootSIFT 归一化+Kmeans+词袋表示+支撑向量机
5. SIFT 特征+Kmeans+词袋表示+集成 SVM 方法
6. 密集 SIFT+RootSIFT 归一化+Kmeans+词袋表示+空间金字塔匹配+集成 SVM 方法

通过对比实验，最终方法 6 取得了最佳性能，分类准确率达到 71.36%，较 baseline 方法提升了 16.51 个百分点。

二、算法流程与原理

算法整体流程为：从图像中提取 SIFT 特征，使用 K-means 聚类构建视觉词典，将特征映射为词袋表示，再训练 SVM 分类器，最后对测试图像应用相同的特征提取和表示过程，使用训练好的分类器进行预测，评估性能。

1. SIFT 特征提取原理

SIFT 是一种在计算机视觉中广泛使用的局部特征描述符，具有对尺度变化、旋转、亮度变化等的不变性。步骤包含尺度空间极值检测、关键点定位、方向赋值、特征描述符生成。除了标准 SIFT，本系统还实现了密集 SIFT(Dense SIFT)，它不依赖于特征点检测，而是在图像上按固定步长均匀采样特征点，这样可以捕获更多的图像信息，特别是对于缺乏明显特征点的区域。

2. 词袋模型与 K-means 原理

词袋模型最初源自文本处理领域，在视觉领域的应用步骤为：从训练图像中提取 SIFT 特征描述符，使用 K-means 对所有特征进行聚类，得到 K 个聚类中心作为“视觉词汇”，将每个特征描述符分配给最近的聚类中心，得到“视觉词”，统计每张图像中各个“视觉词”的出现频率，形成 K 维直方图作为图像的特征表示，对直方图进行归一化处理，消除图像大小和特征数量的影响。词袋模型的优点是简单有效，能够生成固定维度的图像表示，便于后续的分类任务。但缺点是完全忽略了特征的空间关系和几何结构信息。

3. 空间金字塔匹配原理

为了克服词袋模型忽略空间信息的缺点，Lazebnik 等人提出了空间金字塔匹配方法。该方法将图像划分为越来越精细的子区域，并在每个子区域内计算特征的词袋表示。具体来说，在第 ℓ 层，图像被划分为 $2^\ell \times 2^\ell$ 个网格，总共有 L+1 层（从 0 到 L）。空间金字塔匹配的核心思想是，在不同分辨率下进行特征匹配，并根据分辨率的精细程度给予不同的权重。具体而言，匹配的权重与网格大小成反比，在第 ℓ 层的权重为 $\frac{1}{2^{L-\ell}}$ ，这意味着在更精细的层级进行的匹配将获得更高的权重。空间金字塔的最终表示是所有层级的特征直方图的加权组合，这种表示方法既保留了词袋模型的灵活性，又增加了空间布局信息，因此能够提升分类性能。

4. RootSIFT 归一化原理

RootSIFT 是 SIFT 描述符的一种归一化变体，该方法将 SIFT 描述符除以其 L1 范数，使得所有元素的和为 1，对 L1 归一化后的描述符进行元素级的平方根变换。这一变换等价于将 Hellinger 距离用于

SIFT 描述符的比较，相比于原始 SIFT 使用的欧氏距离，Hellinger 距离对于视觉词匹配任务通常有更好的性能

5. SVM 与集成 SVM 分类原理

SVM 是一种有效的监督学习模型，用于分类和回归任务。在本系统中，采用了一对多(one-vs-all)策略实现多类分类，即为每个类别训练一个二分类器，将测试样本分配给得分最高的类别。为了进一步提高性能，本系统还实现了集成 SVM 分类器。集成学习通过组合多个基分类器的预测结果来提高整体性能。具体实现使用了 Bagging 方法，它通过在训练数据的随机子集上训练多个 SVM 分类器，并通过多数投票或平均概率的方式合并预测结果。

三、系统设计与实现

注：本系统环境为 Apple M3 macOS Sequoia 15.0.1，python 环境见 README.md 文件。

1. 系统总体架构

系统采用模块化设计，分为以下几个核心模块：

- **feature_extraction.py**：实现特征提取功能（SIFT，密集 SIFT，RootSIFT）
- **feature_representation.py**：实现特征表示方法（码本构建，词袋模型，空间金字塔匹配）
- **classifier.py**：实现分类器（SVM，集成 SVM）
- **main.py**：主程序，实验配置和执行入口

这种模块化设计便于切换不同的实验配置，易于扩展和比较不同算法的性能。

2. 特征提取模块实现

特征提取模块主要实现了三种特征提取方法：标准 SIFT、密集 SIFT 和 RootSIFT 归一化。以下是核心函数的实现及参数说明：

标准 SIFT 提取：

```
def extract_sift_features(image_path):
    """
    从图像中提取 SIFT 特征
    参数:
        image_path (str): 图像文件的路径
    返回:
        numpy.ndarray 或 None: SIFT 特征描述符矩阵，形状为(N, 128)
    """
    # 读取图像并转换为灰度图
    img = cv2.imread(image_path)
    if img is None:
        print(f"无法读取图像: {image_path}")
        return None
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # 创建 SIFT 检测器并计算描述符
    sift = cv2.SIFT_create()
    _, descriptors = sift.detectAndCompute(gray, None)
    return descriptors
```

密集 SIFT 提取：

```
def extract_dense_sift_features(image_path, step_size=8):
    """
    从图像中提取密集 SIFT 特征
    参数:
        image_path (str): 图像文件的路径
        step_size (int, 可选): 密集采样的步长，默认为 8 像素
    返回:
        numpy.ndarray 或 None: 密集 SIFT 特征描述符矩阵
    """
    # 读取图像并转换为灰度图
    img = cv2.imread(image_path)
    if img is None:
        # ... 处理读取失败
        return None
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # 创建 SIFT 检测器
    sift = cv2.SIFT_create()

    # 创建密集采样点
    h, w = gray.shape
```

```

keypoints = []
for y in range(0, h, step_size):
    for x in range(0, w, step_size):
        # 使用 16x16 像素的图像块和 8 像素的步长
        keypoints.append(cv2.KeyPoint(x, y, 16))

# 计算描述符
_, descriptors = sift.compute(gray, keypoints)
return descriptors

```

密集 SIFT 不依赖于特征点检测，而是在图像上以固定步长（默认为 8 像素）均匀采样点，对每个点计算 SIFT 描述符。这种方法能够捕获更全面的图像信息，特别适合场景分类任务。

RootSIFT 归一化：

```

def apply_root_sift(descriptors):
    """
    应用 RootSIFT 归一化到 SIFT 描述符

    参数:
        descriptors (numpy.ndarray): SIFT 特征描述符矩阵，形状为(N, 128)

    返回:
        numpy.ndarray 或 None:
            - 如果输入有效，返回 RootSIFT 归一化后的特征描述符矩阵，形状与输入相同
            - 如果输入为 None 或空列表，返回 None

    """
    if descriptors is None or len(descriptors) == 0:
        return None
    # L1 归一化
    eps = 1e-7 # 防止除以零
    descriptors = descriptors / (np.sum(descriptors, axis=1, keepdims=True) + eps)
    # 平方根变换
    descriptors = np.sqrt(descriptors)
    return descriptors

```

RootSIFT 归一化通过 L1 归一化和平方根变换处理原始 SIFT 描述符，使用 Hellinger 距离度量代替欧氏距离，这通常能提升特征匹配性能。步长参数 `step_size=8` 和图像块大小 `16x16` 的选择是基于论文中推荐的设置。

3. 特征表示模块实现

特征表示模块主要负责构建词袋模型和空间金字塔表示。关键函数实现如下：

码本构建：

```

def build_codebook(features_list, codebook_size):
    """使用 KMeans 构建词袋模型的码本"""
    # 将所有特征合并成一个大矩阵
    all_features = np.vstack([f for f in features_list if f is not None and len(f) > 0])
    print(f'聚类 {all_features.shape[0]} 个特征到 {codebook_size} 个视觉词...')
    # 使用 MiniBatchKMeans 进行大规模聚类
    if all_features.shape[0] > 100000:
        kmeans = MiniBatchKMeans(n_clusters=codebook_size,
                                  batch_size=1000,
                                  random_state=42)
    else:
        kmeans = KMeans(n_clusters=codebook_size,
                        random_state=42,
                        n_init=10)
    kmeans.fit(all_features)
    return kmeans

```

该函数将所有图像的特征描述符合并，然后使用 K-means 聚类算法将它们聚类为 `codebook_size` 个中心，这些中心构成了视觉词典。当特征数量较大时（超过 10 万个），使用 `MiniBatchKMeans` 以提高效率。码本大小设置为 200 是参考论文后采取的一个折中选择，较小的码本不足以表示丰富的视觉信息，而过大的码本会增加计算复杂度且可能导致过拟合。

词袋模型表示：

```

def compute_bovw_features(image_paths, codebook, dense_sift=False, root_sift=False, step_size=8):
    """
    计算图像的词袋模型(BoVW)特征表示

    参数:
        image_paths (list): 图像路径列表
        codebook (sklearn.cluster.KMeans): 训练好的码本，包含视觉词汇
        dense_sift (bool, 可选): 是否使用密集 SIFT，默认为 False
        root_sift (bool, 可选): 是否应用 RootSIFT 归一化，默认为 False
        step_size (int, 可选): 密集 SIFT 的步长，仅在 dense_sift=True 时使用，默认为 8

    返回:
        numpy.ndarray: 词袋模型特征矩阵，形状为(N, K)，其中 N 是图像数量，
            K 是码本大小 (即 codebook.n_clusters)
    """

```

```

"""
bovw_features = []
for img_path in image_paths:
    # 提取 SIFT 特征
    if dense_sift:
        descriptors = extract_dense_sift_features(img_path, step_size=step_size)
    else:
        descriptors = extract_sift_features(img_path)
    # 应用 RootSIFT
    if root_sift and descriptors is not None and len(descriptors) > 0:
        descriptors = apply_root_sift(descriptors)
    # 计算词袋模型表示
    if descriptors is not None and len(descriptors) > 0:
        # 预测每个特征向量所属的聚类
        visual_words = codebook.predict(descriptors)
        # 计算直方图
        hist, _ = np.histogram(visual_words, bins=codebook.n_clusters,
                                range=(0, codebook.n_clusters-1))

        # 归一化直方图
        if np.sum(hist) > 0:
            hist = hist / np.sum(hist)
        else:
            # 如果没有提取到特征, 则使用零向量
            hist = np.zeros(codebook.n_clusters)
        bovw_features.append(hist)
return np.array(bovw_features)

```

空间金字塔表示:

```

def compute_spatial_pyramid_features(image_paths, codebook, levels=2, dense_sift=False, root_sift=False,
step_size=8):
    """
    计算图像的空间金字塔匹配特征表示

    参数:
        image_paths (list): 图像路径列表
        codebook: 训练好的码本
        levels (int): 金字塔层数, 默认为 2
        dense_sift (bool): 是否使用密集 SIFT
        root_sift (bool): 是否应用 RootSIFT 归一化
        step_size (int): 密集 SIFT 的步长

    返回:
        numpy.ndarray: 空间金字塔特征矩阵
    """
    spatial_pyramid_features = []

    for img_path in image_paths:
        # 读取图像并提取特征
        img = cv2.imread(img_path)
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        h, w = gray.shape
        # 提取特征并记录关键点位置
        if dense_sift:
            # ... 创建密集采样点
            keypoints = []
            for y in range(0, h, step_size):
                for x in range(0, w, step_size):
                    keypoints.append(cv2.KeyPoint(x, y, step_size))

            sift = cv2.SIFT_create()
            _, descriptors = sift.compute(gray, keypoints)
            kp_locations = np.array([[kp.pt[0], kp.pt[1]] for kp in keypoints])
        else:
            # ... 使用标准 SIFT
            sift = cv2.SIFT_create()
            keypoints, descriptors = sift.detectAndCompute(gray, None)
        # 应用 RootSIFT (如果需要)
        if root_sift and descriptors is not None and len(descriptors) > 0:
            descriptors = apply_root_sift(descriptors)
        # 预测视觉词
        visual_words = codebook.predict(descriptors)
        # 计算空间金字塔表示
        pyramid_features = []
        # 基于参考文献中的权重方案
        weights = [1.0 / (2 ** (levels - 1)) for l in range(levels + 1)]
        # 构建金字塔
        for level in range(levels + 1):
            num_bins = 2 ** level
            bin_w = w / num_bins

```

```

bin_h = h / num_bins

# 遍历每个网格
for i in range(num_bins):
    for j in range(num_bins):
        # 计算当前网格的边界
        x_min = i * bin_w
        x_max = (i + 1) * bin_w
        y_min = j * bin_h
        y_max = (j + 1) * bin_h

        # 找出在当前网格内的特征点
        indices = np.where(
            (kp_locations[:, 0] >= x_min) &
            (kp_locations[:, 0] < x_max) &
            (kp_locations[:, 1] >= y_min) &
            (kp_locations[:, 1] < y_max)
        )[0]

        # 计算该网格内的视觉词直方图
        # ... 计算并加权直方图

        # 将直方图添加到金字塔特征
        pyramid_features.append(hist)
# 将所有网格的特征拼接并归一化
final_feature = np.concatenate(pyramid_features)
spatial_pyramid_features.append(final_feature)

return np.array(spatial_pyramid_features)

```

空间金字塔特征表示的计算过程更为复杂。首先，它在多个层级上将图像划分为不同大小的网格；然后，在每个网格中计算视觉词直方图；最后，按照论文中的权重方案对不同层级的直方图进行加权并拼接，形成最终的特征表示。金字塔层数设置为 2（加上原始级别，共 3 层）是一个常用选择，这意味着图像被分为 1×1、2×2 和 4×4 共 21 个区域。加权方案采用原论文中推荐的设置，即权重与网格大小成反比。

4. 分类器模块实现

分类器模块实现了标准 SVM 和集成 SVM 两种分类器：

集成 SVM：

```

def train_svm_classifier(X_train, y_train, C=10, kernel='rbf'):
    """
    训练 SVM 分类器

    参数:
        X_train (numpy.ndarray): 训练特征矩阵，形状为(N, D)，其中 N 是样本数量，D 是特征维度
        y_train (list 或 numpy.ndarray): 训练标签，长度为 N
        C (float, 可选): SVM 的惩罚参数，控制正则化强度，默认为 10
        kernel (str, 可选): 核函数类型，可选项: 'linear', 'poly', 'rbf', 'sigmoid', 默认为'rbf'

    返回:
        tuple: (分类器, 特征缩放器), 包含:
            - sklearn.svm.SVC: 训练好的 SVM 分类器
            - sklearn.preprocessing.StandardScaler: 用于特征标准化的缩放器
    """
    # 特征缩放
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    # 训练 SVM 分类器
    clf = SVC(C=C, kernel=kernel, probability=True, random_state=42)
    clf.fit(X_train_scaled, y_train)
    return clf, scaler

def train_ensemble_svm(X_train, y_train, C=10, kernel='rbf', n_estimators=5):
    """
    训练集成 SVM 分类器

    参数:
        X_train (numpy.ndarray): 训练特征矩阵，形状为(N, D)，其中 N 是样本数量，D 是特征维度
        y_train (list 或 numpy.ndarray): 训练标签，长度为 N
        C (float, 可选): SVM 的惩罚参数，控制正则化强度，默认为 10
        kernel (str, 可选): 核函数类型，可选项: 'linear', 'poly', 'rbf', 'sigmoid', 默认为'rbf'
        n_estimators (int, 可选): 集成学习中基分类器的数量，默认为 5

    返回:
        tuple: (集成分类器, 特征缩放器), 包含:
            - sklearn.ensemble.BaggingClassifier: 训练好的集成 SVM 分类器
            - sklearn.preprocessing.StandardScaler: 用于特征标准化的缩放器
    """

```

```
# 特征缩放
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# 创建基分类器
base_clf = SVC(C=C, kernel=kernel, probability=True, random_state=42)

# 创建集成分类器
ensemble_clf = BaggingClassifier(
    estimator=base_clf,
    n_estimators=n_estimators,
    max_samples=0.8,
    max_features=0.8,
    bootstrap=True,
    bootstrap_features=False,
    random_state=42
)

# 训练集成分类器
ensemble_clf.fit(X_train_scaled, y_train)
return ensemble_clf, scaler
```

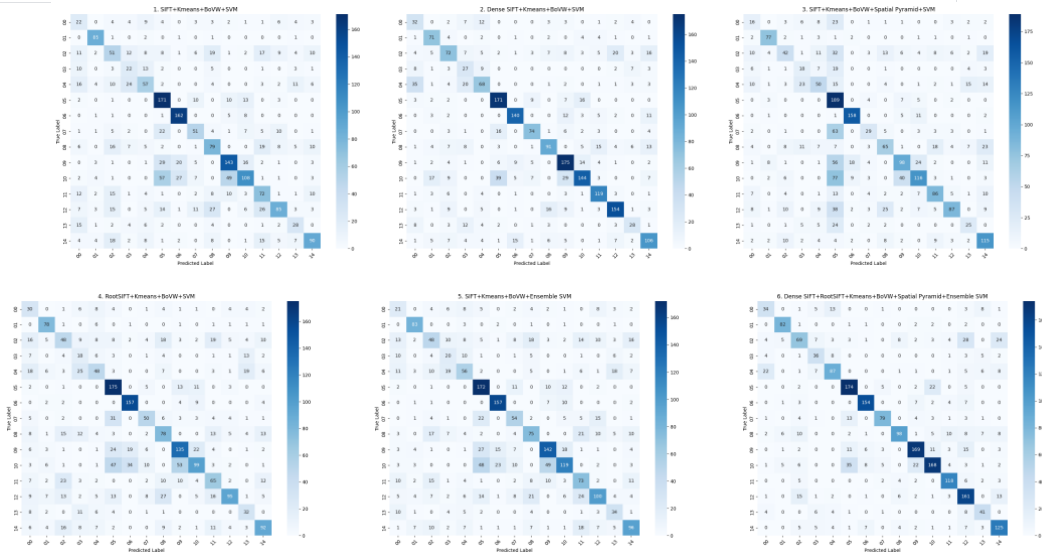
集成 SVM 使用 Bagging 方法，默认使用 5 个基分类器，每个分类器使用训练数据的 80% 和特征的 80% 训练。这些设置有助于增加分类器的多样性，提高整体性能和鲁棒性。

四、系统性能与效果

1. 不同配置下的性能对比

通过实验，对比了六种不同配置下的分类性能：

实验配置	准确率
1. SIFT+Kmeans+BoVW+SVM	0.5485
2. Dense SIFT+Kmeans+BoVW+SVM	0.6586
3. SIFT+Kmeans+BoVW+Spatial Pyramid+SVM	0.5239
4. RootSIFT+Kmeans+BoVW+SVM	0.5369
5. SIFT+Kmeans+BoVW+Ensemble SVM	0.5593
6. Dense SIFT+RootSIFT+Kmeans+BoVW+Spatial Pyramid+Ensemble SVM	0.7136



从结果来看，有以下几个重要发现：

- 密集 SIFT 显著优于标准 SIFT**：配置 2 比配置 1 提升了 11.01 个百分点，这表明在场景分类任务中，密集采样的特征比稀疏特征更有效，能够捕获更全面的场景信息。
- RootSIFT 归一化略有帮助**：配置 4 比配置 1 略有提升，说明 RootSIFT 归一化对于改善特征匹配质量有一定帮助。
- 集成 SVM 提升有限**：配置 5 比配置 1 提升了 1.08 个百分点，说明单纯改进分类器的收益有限。
- 不同场景类别分类难度存在差异**：通过混淆矩阵可以清晰地观察到不同场景类别的分类难度差异。具体来说，00（卧室）和 04（客厅）、05（海边）和 10（山地）和 09（感觉也是山地？）常被混淆。
- 组合方法效果最佳**：配置 6 综合了密集 SIFT、RootSIFT 归一化、空间金字塔和集成 SVM 的优势，取得了最高的准确率 71.36%，比基准方法提升了 16.51 个百分点。